

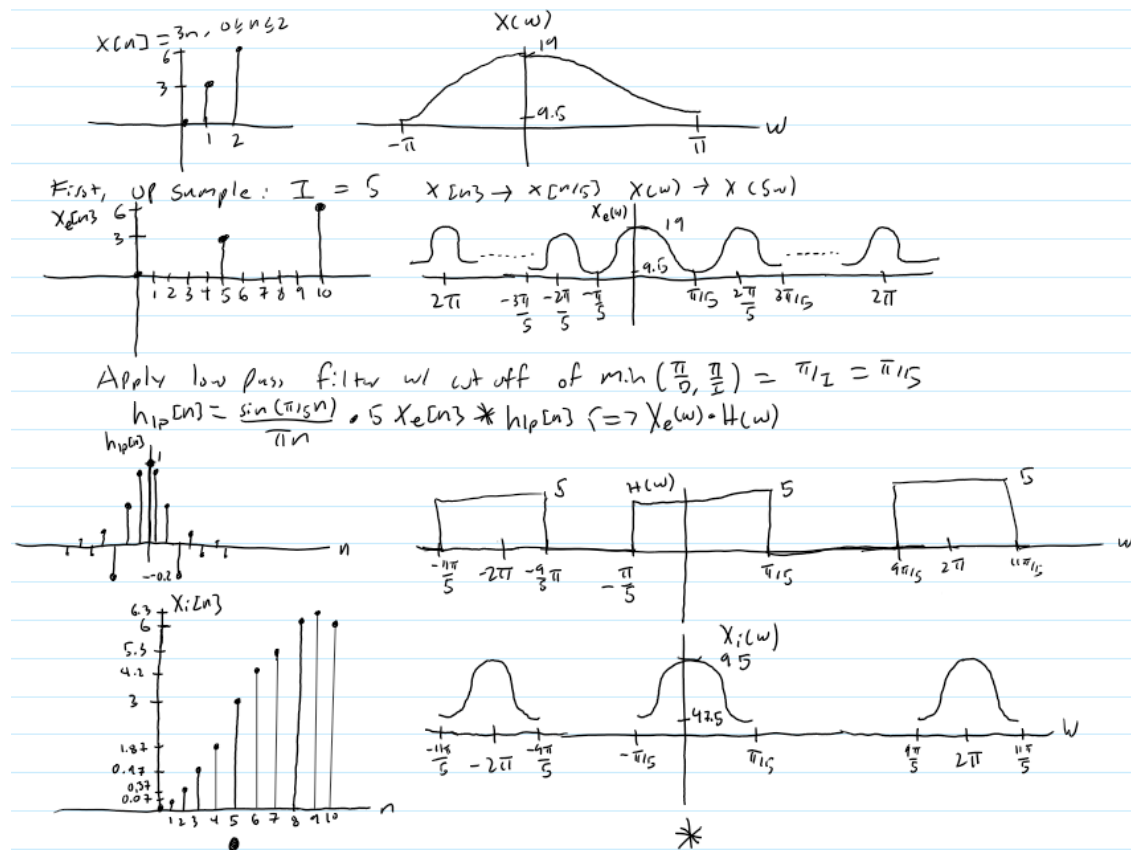
Project 2 – EECS 152B

Cameron Peterson-Zopf

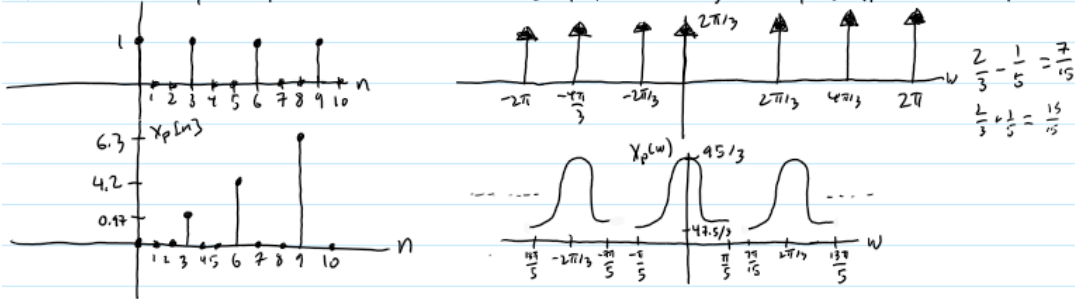
Question 1: Sample Rate Conversion of Random Signal

For this question, students take a signal whose DTFT occupies the entire spectrum of $-\pi$ to π and performs sample rate conversions of $5/3$ and $3/5$. I will explain the $5/3$ sample rate conversion first and then $3/5$. Both the time signals and their corresponding spectra will be plotted for each step.

Beginning with the $5/3$ conversion, we start with $x[n]$ and $X(\omega)$, then up sample by 5, apply the convolution of the interpolation filter with the down sampling filter, then down sample by 3. When down sampling, first a delta train is applied and then the horizontal axis is compressed. These results are displayed below in figure 1.



Now, this above is the interpolated results. From here, we will now decimate the signal by a factor of three. Multiply time signal by delta train of period D



Now we do a compression in the time domain to get rid of zero pts

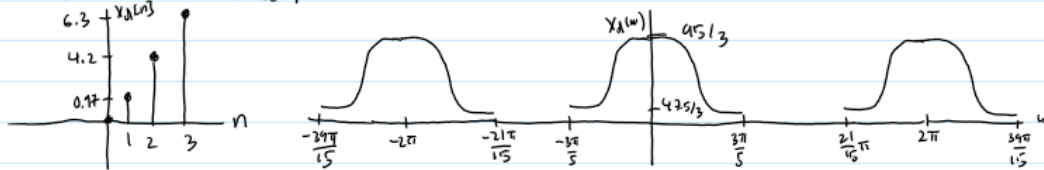
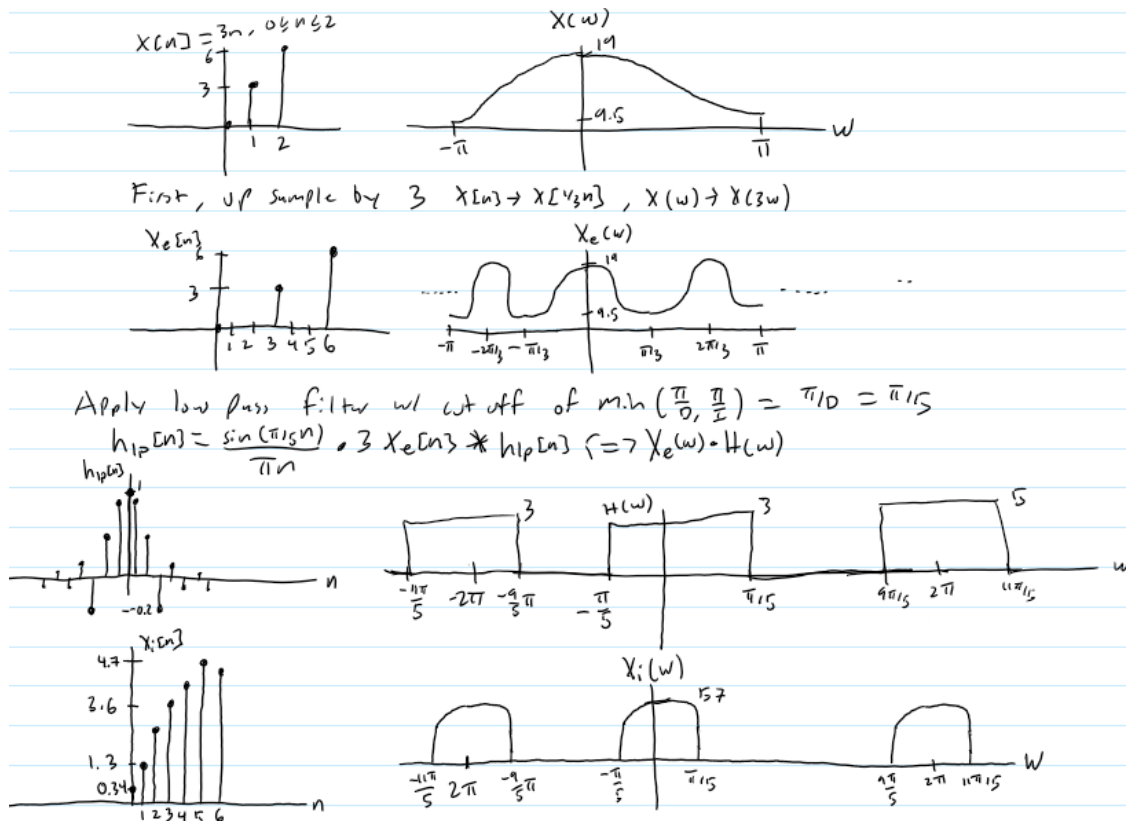


Figure 1 – Sample Rate Conversion by 5/3

Next, the same sequence of steps is applied for the sample rate conversion of 3/5. The only difference is that now the convolution of the interpolating filter and the down sampling filter will be the down sampling filter and thus there will be distortion as seen in figure 2.



We see there is distortion. The time domain values are slightly different due to the distortion from the low pass filter.

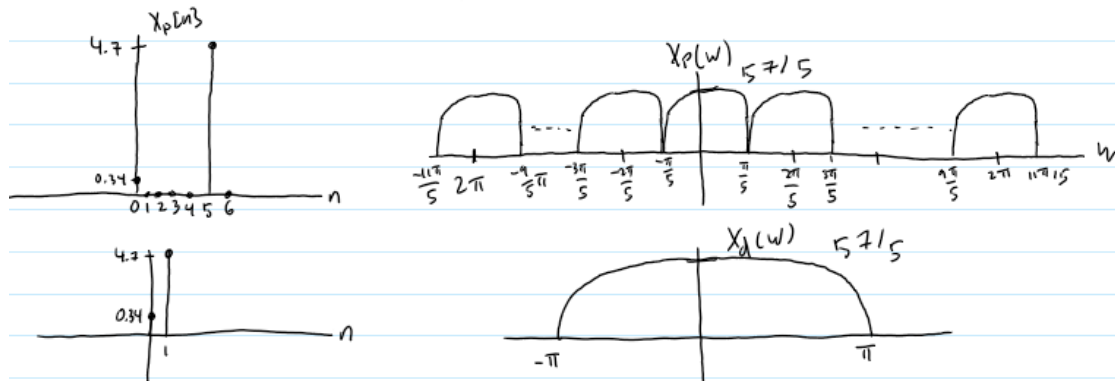
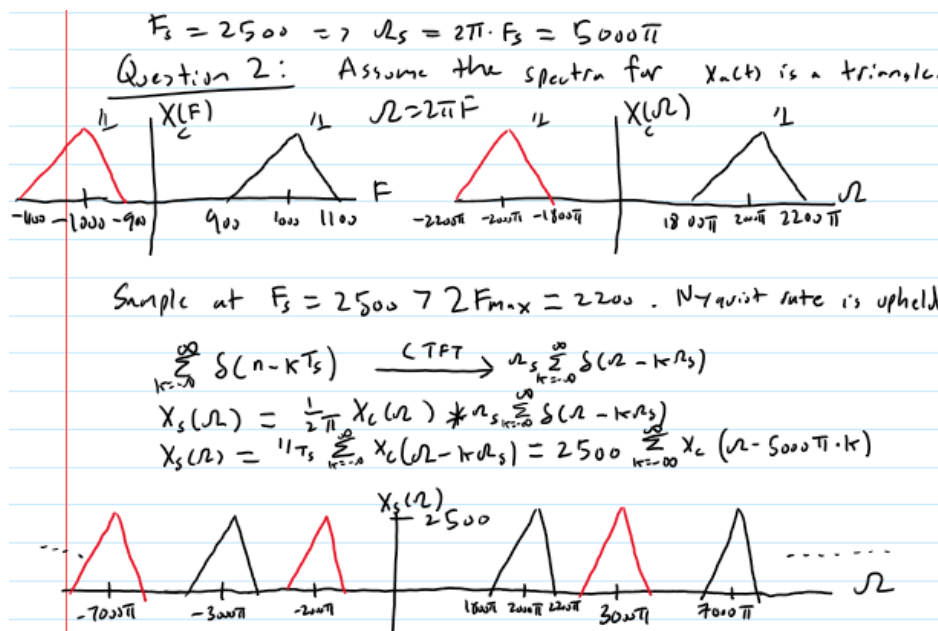


Figure 2 – Sample Rate Conversion of 3/5

Question 2 – Bandlimited Signal

This question analyses a band limited signal as it goes through a system. Assuming the signal is real, the magnitude of the signal will be even in the frequency domain. The spectra for each step through the system is displayed in figure 3.



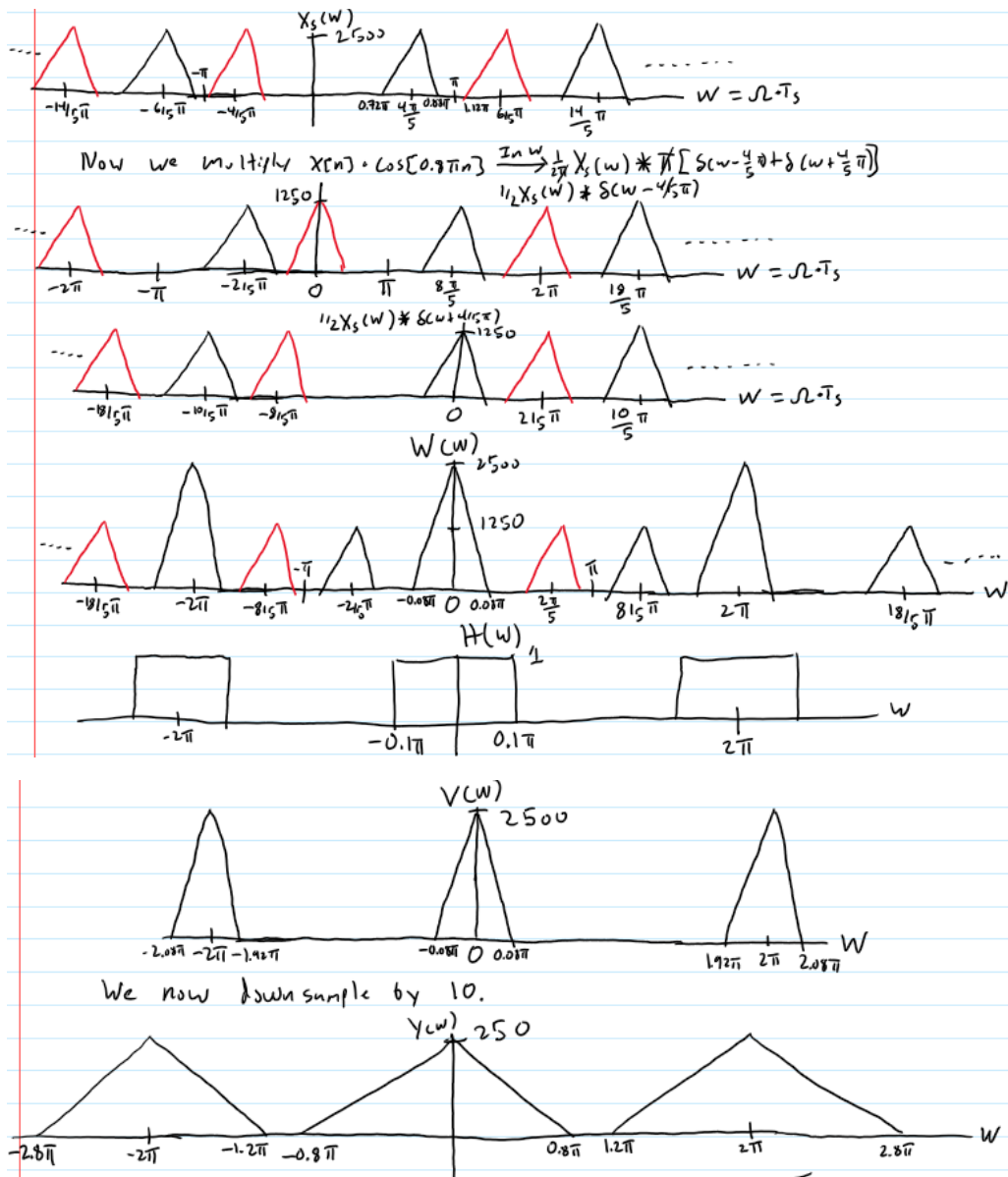


Figure 3 – Analysis of Bandlimited Signal

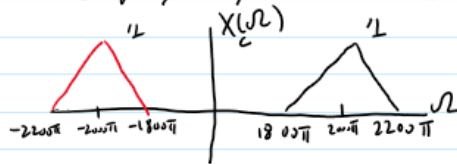
Then, the second part of the question asks students if it is possible to obtain $y(n)$ by sampling $x_a(t)$ with a period of $T = 4\text{ms}$, which is 10 times faster than the previous sampling rate. These results are displayed in figure 4.

$$2B) T_s = 4 \text{ ms} \quad F_{\max} = 1100 \text{ Hz}$$

$$F_s = 250 \text{ Hz}$$

To uphold Nyquist rate, $F_s > 2 \cdot F_{\max}$

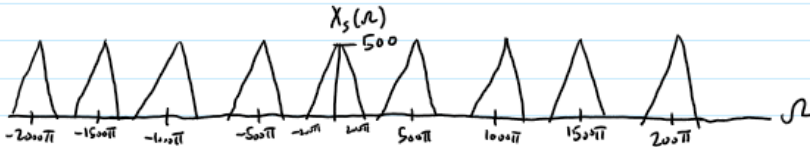
$250 < 2200 \text{ Hz}$, thus we will not be greater than the Nyquist rate. However, we are dealing w/ a bandlimited signal. Since we have shifted the original signal by $\pm 1000 \text{ Hz}$, which is a multiple of the sampling rate, and the bandwidth is less than the sampling rate, then the output may be recoverable.



$$\sum_{k=-\infty}^{\infty} \delta(n - kT_s) \xrightarrow{\text{CTFT}} \frac{1}{T_s} \sum_{k=-\infty}^{\infty} \delta(\omega - k\omega_s)$$

$$X_s(\omega) = \frac{1}{T_s} X_c(\omega) * \sum_{k=-\infty}^{\infty} \delta(\omega - k\omega_s)$$

$$X_s(\omega) = \frac{1}{T_s} \sum_{k=-\infty}^{\infty} X_c(\omega - k\omega_s) = 250 \sum_{k=-\infty}^{\infty} X_c(\omega - 500\pi \cdot k)$$



The red and black triangles will align exactly on top of each other and double the heights.

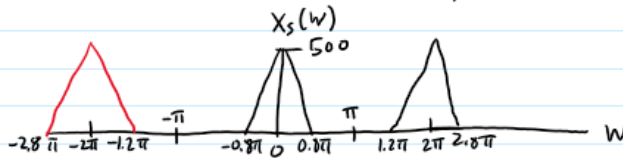


Figure 4 – Sampling Bandlimited Signal at 250 Hz.

We see that while we are under the sampling rate, because the copies of the triangles land perfectly on top of each other, we are able to the correct copy of our signal, just scaled by a factor of 2.

Question 3 – 1000 Hz Sine Wave Sampled at 20 kHz

This question prompts students to sample a 1000 Hz sine wave at 20 kHz. Then, the students will play this frequency sound to obtain a reference point for further questions. The code for this question is displayed in figure 5.

```

Editor - C:\Users\camer\Documents\MATLAB\Project 2\Project2Q3.m
dtfft.m  project1.m  Project2Q1.m  Project2Q3.m
1  %x = sin(2*pi*1000*t);
2  %this would be my signal in time
3  n = 0:1000;
4  %Fs = 20 kHz, Ts =
5  x = sin(2*pi*1000*n*5*10^(-5));
6  %x = sin(2*pi*n/20);
7  sound(x, 20000);
8

```

Figure 5 – Code for Question 3

Question 4 – 1000 Hz Sine Wave Sampled at 100 kHz

Question 4 instructs students to up sample the signal from question 3 by 5 to 100 kHz. Playing the signal after inserting the zeros, and then after performing the interpolation, I noticed that the signals sounded roughly the same. The waveforms for the original signal and the up sampled signal are shown in figure 7 and 8 respectively, with the code for the question in figure 6.

```

Editor - C:\Users\camer\Documents\MATLAB\Project2Q4.m
dtfft.m  project1.m  Project2Q1.m  Project2Q3.m  Project2Q4.m
1  %x = sin(2*pi*1000*t);
2  %this would be my signal in time
3  n = 0:1000; %size [1 1001]
4  ni = 0:5105; %size [1 5106]
5  %Fs = 20 kHz
6  %x = sin(2*pi*1000*n*5*10^(-5));
7  x = sin(2*pi*n/20); %size [1 1001]
8  %up sample to 100 kHz => I = 5
9  Z = [x;zeros(4,1001)]; % add 4 zeros in between each sample
10 z = Z(:); % convert to column vector
11 xe = z'; % convert to row vector
12 %xe is the up sampled version
13 hlp = firpm(101,[0 0.2 0.3 1],[5 5 0 0]);
14 xi = conv(hlp, xe);
15 %xi is size [1 5106]
16 %this is the interpolated signal
17
18 %Plot 20 kHz Sampled Signal
19 figure(1);
20 plot(n/20000,x);
21 title('Fs = 20 kHz');
22 xlabel('Time (secs)');
23 ylabel('Amplitude');
24 %Plot 100 kHz Sampled Signal
25 figure(2)
26 plot(ni/100000,xi);
27 title('Fs = 100 kHz');
28 xlabel('Time (secs)');
29 ylabel('Amplitude');
30 sound(xe, 100000);
31 %sound after inserting zeros
32 sound(xi, 100000);
33 %sound after interpolation
34

```

Figure 6 – Code for Question 4

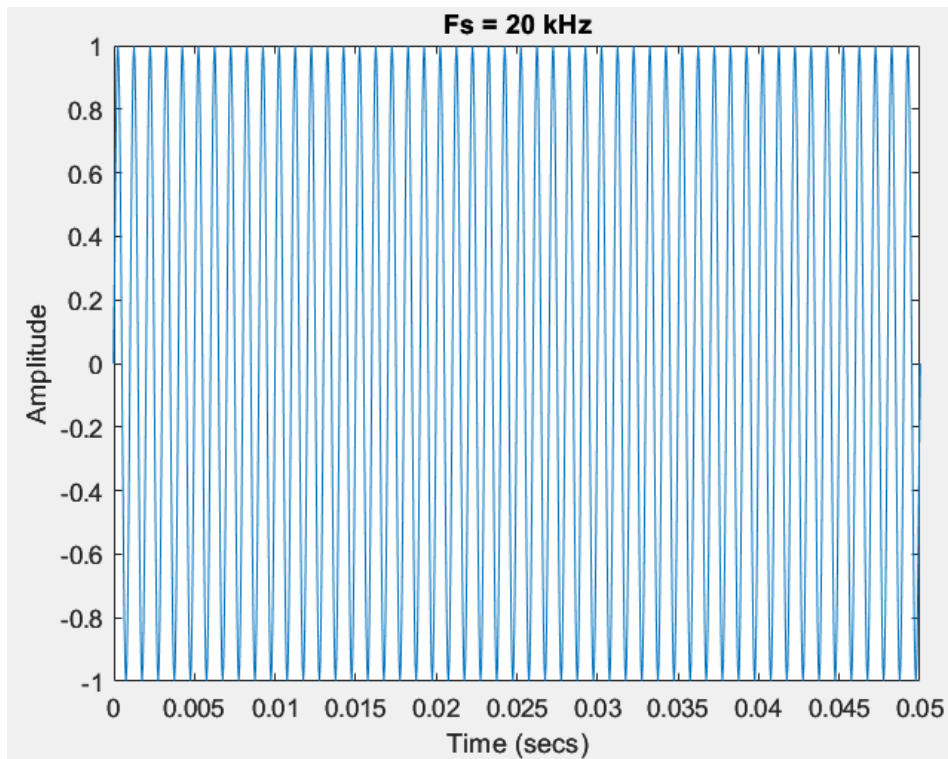


Figure 7 - Waveform with Sampling of 20 kHz

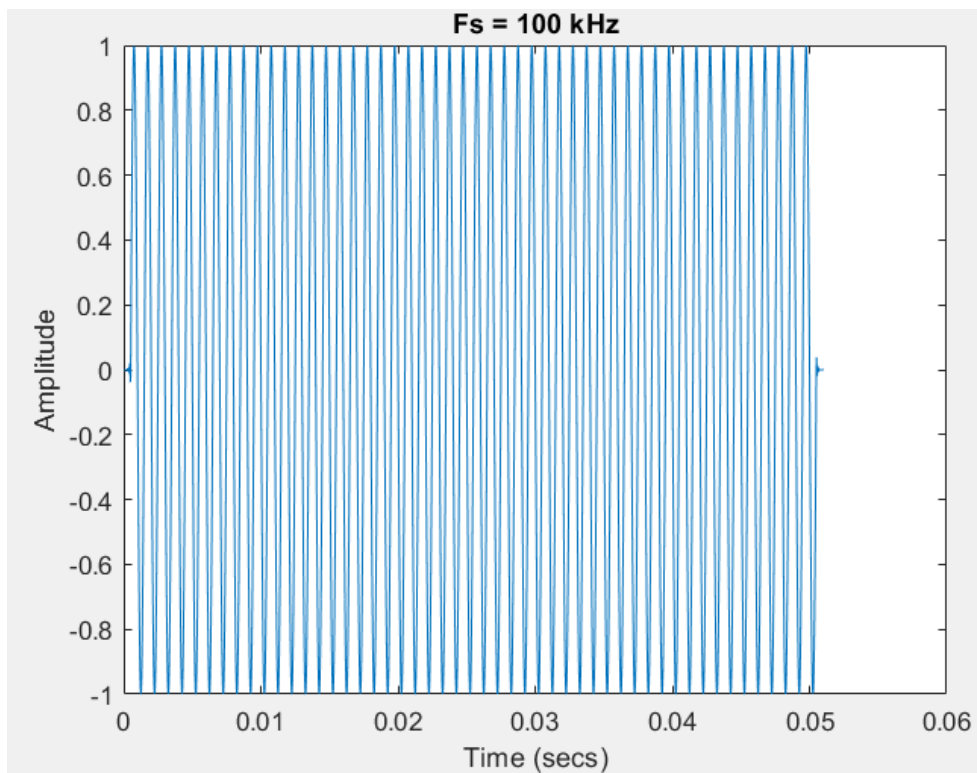


Figure 8 – Waveform with Sampling of 100 kHz

Zooming in on the two graphs in figure 9, we see that the higher sampled signal has smoother curvature. A prime example is at the peak values, where we see that at 20 kHz the peaks are triangular, whereas for 100 kHz they are much more curved and hence better represent the sine wave.

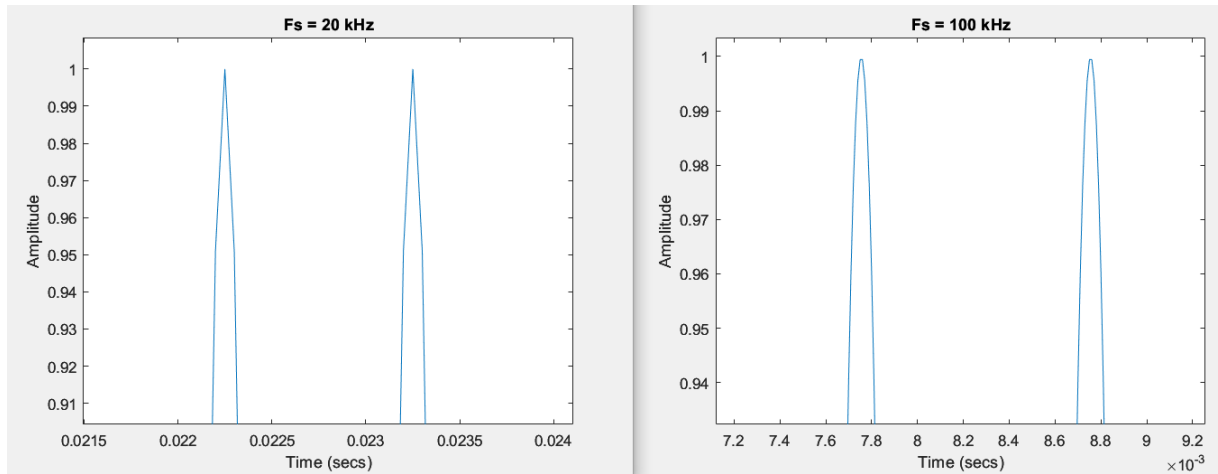


Figure 9 – Zoomed in Comparison of Figures 7 and 8.

As expected, both waveforms have roughly the same curvature, as both are sampling the same signal, and both are above the Nyquist rate (both are greater than 2000 kHz). Thus, the only difference is in the smoothness of the curves, which the higher sampled one would have an advantage.

Question 5 – Down Sampling a 1000 Hz Sine Wave

Question 5 asks students to down sample the signal from question 3 by a factor of 5 down to 4 kHz. The Nyquist rate states that to obtain a correct representation of the signal we must sample at a rate twice as fast as the highest frequency in the signal. In this case, we must sample higher than 2 kHz. Thus, reducing the sampling rate by 5 to 4kHz is still above the minimum rate needed and thus the anti-aliasing filter will not affect anything. Refer to figure 7 for the originally sampled signal. Figure 10 shows the code utilized for question 5.


```
Editor - C:\Users\camer\Documents\MATLAB\Project2Q5.m
dtft.m x project1.m x Project2Q1.m x Project2Q3.m
1  n = 1:1000; %size [1 1000]
2  x = sin(2*pi*n/20); %size [1 1000]
3
4  %Plot 20 kHz Sampled Signal
5  figure(1);
6  plot(n/20000,x);
7  title('Fs = 20 kHz');
8  xlabel('Time (secs)');
9  ylabel('Amplitude');
10
11 %Downsample w/ no Filter
12 xd = downsample(x,5);
13 nd = 1:200; %downsample by 5 thus 1/5 # of pts
14 figure(2);
15 plot(nd/4000,xd);
16 title('4 kHz Sampling w/ No Filter');
17 xlabel('Time (secs)');
18 ylabel('Amplitude');
19
20 %Downsample w/ Filter
21 hlp = firpm(101,[0 0.2 0.3 1],[1 1 0 0]);
22 c = conv(hlp,x);
23 %size [1 1101]
24 xd2 = downsample(c,5);
25 %size [1 221]
26 nd2 = 1:221;
27 figure(3);
28 plot(nd2/4000,xd2);
29 title('4 kHz Sampling w/ Filter');
30 xlabel('Time (secs)');
31 ylabel('Amplitude');
32
33 sound(x,20000);
34 pause(3);
35 sound(xd,4000);
36 pause(3);
37 sound(xd2,4000);
```

Figure 10 – Code for Question 5

The sound for all three cases sounds exactly the same as expected since we are above the Nyquist rate for every case. The waveforms are shown below, beginning with the down sampled signal without a filter in figure 11.

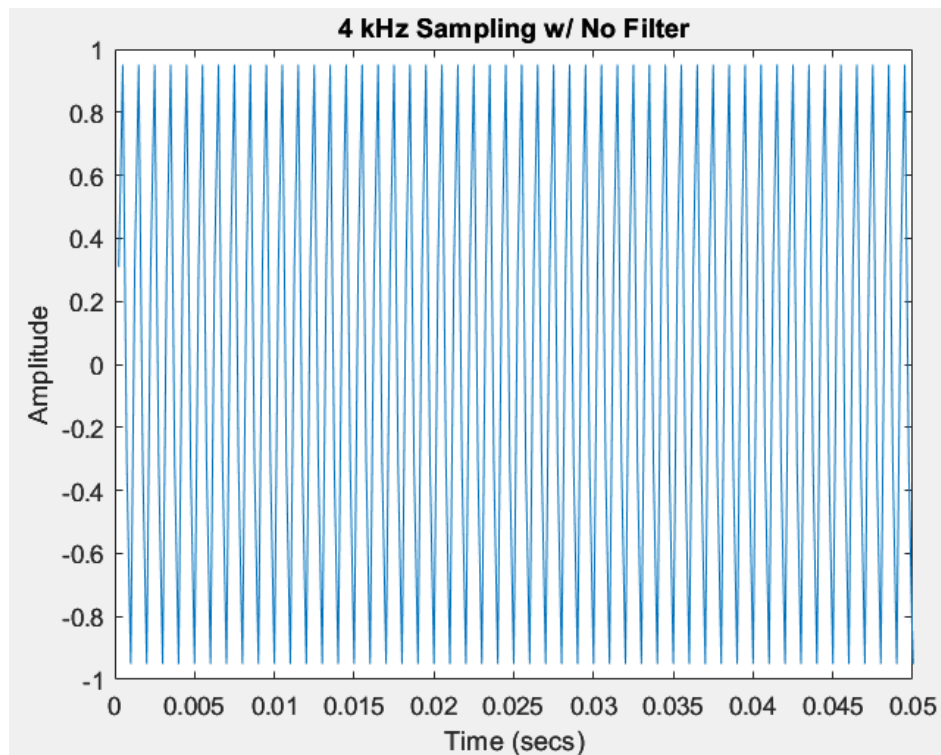


Figure 11 – Down sampled Signal with no Filter

Figure 12 shows the down sampled signal with a filter.

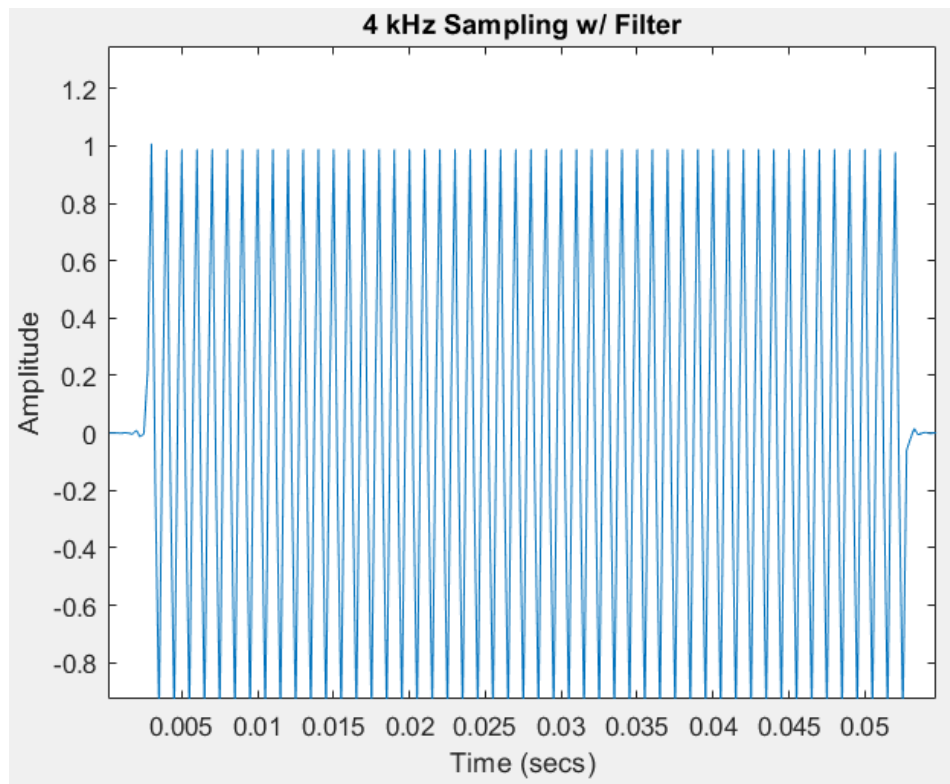


Figure 12 – Down sampled Signal with Filter

Comparing figures 11 and 12, the signal with the filter has small sections at the beginning and end with an amplitude of zero. Figure 13 compares the peaks of the two filters.

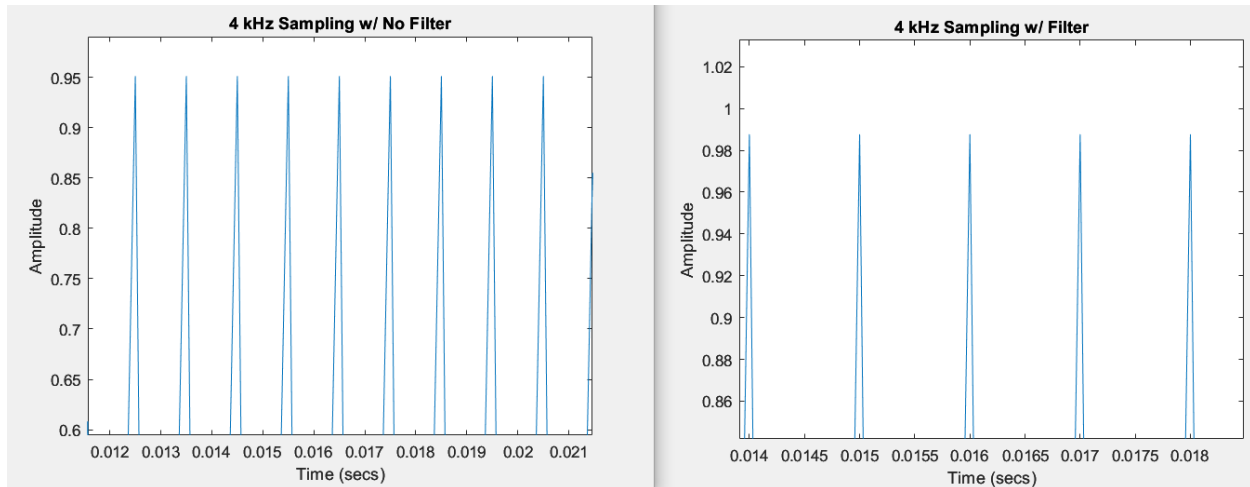


Figure 13 – Zoomed in Comparison

Both curves have roughly the same smoothness; however, the sampled signal has a slightly higher amplitude.

Question 6 – Down Sampling Sine Wave by 12

Question 6 has students down sample the sine wave by a factor of 12, corresponding to a sampled frequency of 1.67 kHz. Notice that now we have down sampled to a frequency lower than the Nyquist rate, which is at 2 kHz. Thus, the sound should be of a different tone compared to the original. If we had utilized a filter then the signal would have been distorted, but because we are not, then the signal will have aliasing. More specifically, the sampling frequency of 1.67 kHz will only be able to successfully interpret signals of half of its frequency, or below 833 Hz. Frequencies above 833 Hz will be aliased. Thus, we should hear a sound of lower frequency. The code for this question is displayed in figure 14.

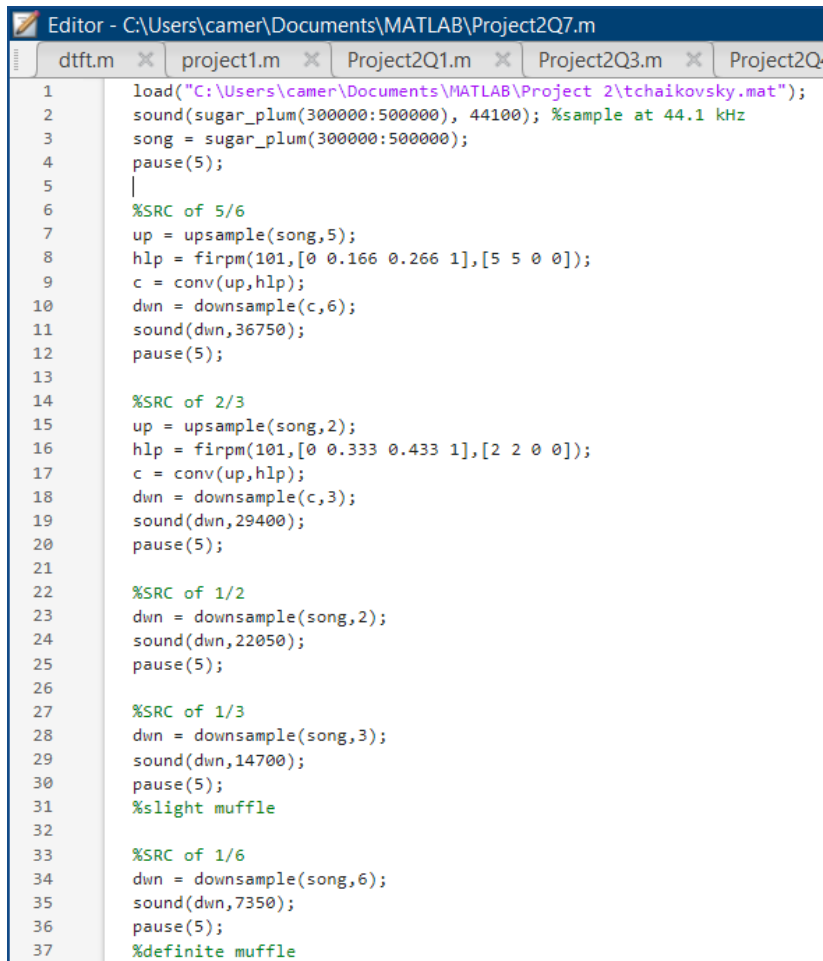
```
Editor - C:\Users\camer\Documents\MATLAB\Project2Q6.m
+3 Project2Q3.m Project2Q4.m Project2Q5.m
1 n = 1:1000; %size [1 1000]
2 x = sin(2*pi*n/20); %size [1 1000]
3
4 %Downsample w/ no Filter
5 xd = downsample(x,12); %size [1 84]
6 size(xd)
7 nd = 1:84;
8 sound(xd,1667);
```

Figure 14 – Code for Question 6

Upon listening to the new sound and comparing it to the original sample's sound, the new sound is lower than the original, as expected.

Question 7 – Dance of the Sugar Plum Fairy

For this question students analyze the dance of the sugar plum fairy for different frequencies. The first part of the question asks students to have the following sample rate conversions: $5/6$, $2/3$, $1/2$, $1/3$, and $1/6$. The conversions for the last three will not have the low pass filter. For all of these questions, only a part of the entire song is played. Figure 15 displays the code utilized for this section of the question.

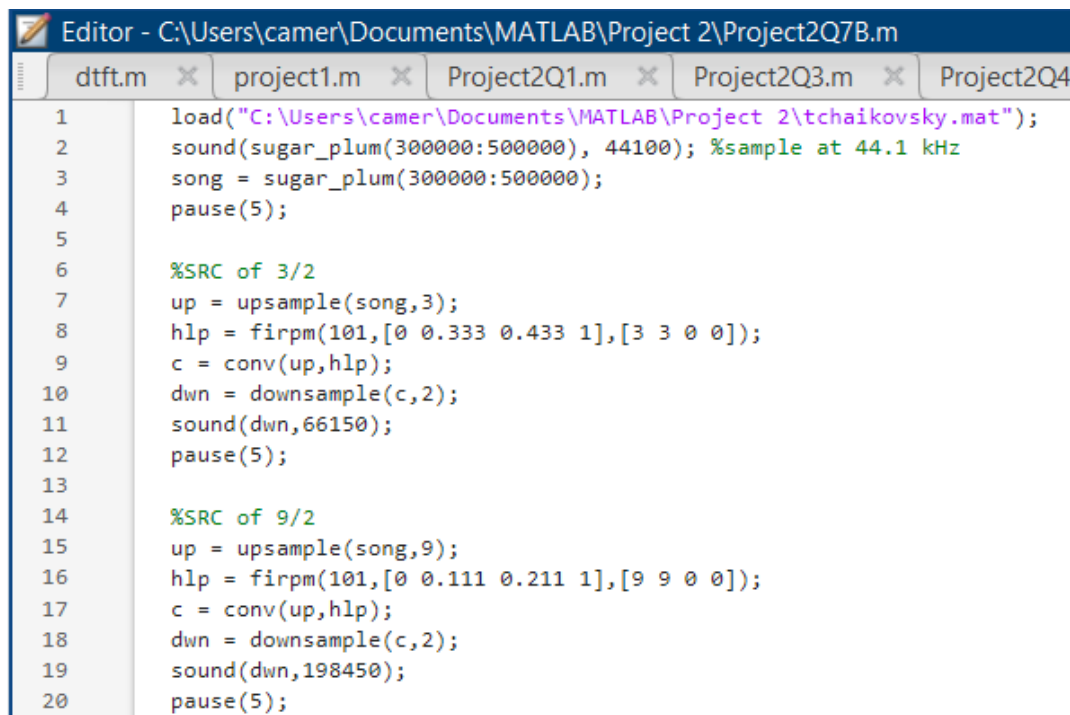


```
Editor - C:\Users\camer\Documents\MATLAB\Project2Q7.m
dtfft.m x project1.m x Project2Q1.m x Project2Q3.m x Project2Q
1 load("C:\Users\camer\Documents\MATLAB\Project 2\tchaikovsky.mat");
2 sound(sugar_plum(300000:500000), 44100); %sample at 44.1 kHz
3 song = sugar_plum(300000:500000);
4 pause(5);
5 |
6 %SRC of 5/6
7 up = upsample(song,5);
8 hlp = firpm(101,[0 0.166 0.266 1],[5 5 0 0]);
9 c = conv(up,hlp);
10 dwn = downsample(c,6);
11 sound(dwn,36750);
12 pause(5);
13
14 %SRC of 2/3
15 up = upsample(song,2);
16 hlp = firpm(101,[0 0.333 0.433 1],[2 2 0 0]);
17 c = conv(up,hlp);
18 dwn = downsample(c,3);
19 sound(dwn,29400);
20 pause(5);
21
22 %SRC of 1/2
23 dwn = downsample(song,2);
24 sound(dwn,22050);
25 pause(5);
26
27 %SRC of 1/3
28 dwn = downsample(song,3);
29 sound(dwn,14700);
30 pause(5);
31 %slight muffle
32
33 %SRC of 1/6
34 dwn = downsample(song,6);
35 sound(dwn,7350);
36 pause(5);
37 %definite muffle
```

Figure 15 – Code for Sample Rate Conversions of $5/6$, $2/3$, $1/2$, $1/3$, and $1/6$

The first three sound exactly the same; however, the $1/3$ appears to be slightly muffled and the $1/6$ is considerably muffled. Thus, we see that without the anti-aliasing filter, we will lose the higher frequency components once we down sample to less than the Nyquist rate.

The second part of the question involves students sampling the song at $3/2$ and $9/2$ of the original rate. Since we are sampling above the Nyquist rate the entire time, no differences should be present in the sounds. The code for this part is shown in figure 16.

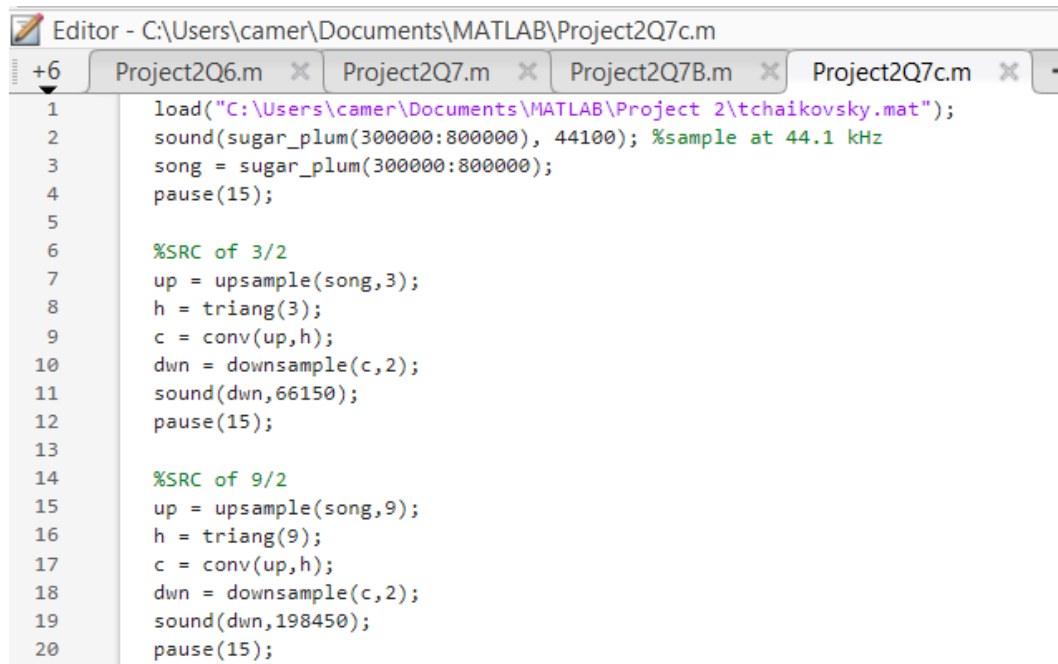
The image shows a MATLAB Editor window with the title bar "Editor - C:\Users\camer\Documents\MATLAB\Project 2\Project2Q7B.m". The window contains several tabs: "dtft.m", "project1.m", "Project2Q1.m", "Project2Q3.m", and "Project2Q4". The active tab is "Project2Q3.m", which displays the following MATLAB code:

```
1 load("C:\Users\camer\Documents\MATLAB\Project 2\tchaikovsky.mat");
2 sound(sugar_plum(300000:500000), 44100); %sample at 44.1 kHz
3 song = sugar_plum(300000:500000);
4 pause(5);
5
6 %SRC of 3/2
7 up = upsample(song,3);
8 hlp = firpm(101,[0 0.333 0.433 1],[3 3 0 0]);
9 c = conv(up,hlp);
10 dwn = downsample(c,2);
11 sound(dwn,66150);
12 pause(5);
13
14 %SRC of 9/2
15 up = upsample(song,9);
16 hlp = firpm(101,[0 0.111 0.211 1],[9 9 0 0]);
17 c = conv(up,hlp);
18 dwn = downsample(c,2);
19 sound(dwn,198450);
20 pause(5);
```

Figure 16 – Code for Sample Rate Conversions of 3/2 and 9/2

As expected, the sounds all sound identical, as all are above the Nyquist rate.

The final aspect of this report involves sample rate conversions of 3/2 and 9/2 with linear interpolation. Linear interpolation is most accurate when the distance between samples is smallest, and hence when we sample the fastest. Thus, the rate of 9/2 will be more accurate than the 3/2, if there is any noticeable difference at all. The code for this section is shown in figure 17.

The image shows a MATLAB Editor window with the title bar 'Editor - C:\Users\camer\Documents\MATLAB\Project2Q7c.m'. There are four tabs open: 'Project2Q6.m', 'Project2Q7.m', 'Project2Q7B.m', and 'Project2Q7c.m'. The 'Project2Q7c.m' tab is active, showing a script with 20 lines of code. The code starts with loading a file 'tchaikovsky.mat' and playing a sound at 44.1 kHz. It then performs two sample rate conversions: first by a factor of 3/2, and then by a factor of 9/2. Each conversion involves upsampling, filtering with a triangular kernel, and downsampling. The code is as follows:

```
1 load("C:\Users\camer\Documents\MATLAB\Project 2\tchaikovsky.mat");
2 sound(sugar_plum(300000:800000), 44100); %sample at 44.1 kHz
3 song = sugar_plum(300000:800000);
4 pause(15);
5
6 %SRC of 3/2
7 up = upsample(song,3);
8 h = triang(3);
9 c = conv(up,h);
10 dwn = downsample(c,2);
11 sound(dwn,66150);
12 pause(15);
13
14 %SRC of 9/2
15 up = upsample(song,9);
16 h = triang(9);
17 c = conv(up,h);
18 dwn = downsample(c,2);
19 sound(dwn,198450);
20 pause(15);
```

Figure 16 – Code for Sample Rate Conversions with Linear Interpolation

I increased the length of the song played in this section, because with the previous length I did not hear a difference. Increasing the amount of playing time also did not achieve any difference in song. Linear sampling is accurate when the original sampling rate is much higher than the Nyquist rate, because then, the samples will be closely spaced and hence not vary significantly. Then, the linear approximation is reasonable due to the small variations between samples. When the original sampling rate is smaller, closer to the Nyquist rate, then the samples will be more spaced out and may vary a lot in amplitude. Hence, linear sampling will not be as accurate in this case. Therefore, we see that with a sampling rate of conversion of $3/2$, we were already above the Nyquist rate to the point where the linear interpolation was accurate. Then, increasing the sampling rate to $9/2$ would make no difference as the $3/2$ was already accurate. Hence, both signals sound just like the original.